L12: Generative Deep Learning

CSci 560 Deep Learning w/ Python (Chollet) Ch. 12

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L12.1 Text Generation

CSci 560: Deep Learning w/ Python (Chollet) Ch. 12.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

How do you Generate Sequence Data?

- Train a model (usually Transformer or RNN) to predict next token
  - e.g. like the encoder / decoder architecture, though we only have 1 sequence so only need decoder
- Any network that can model the probability of the next token given the previous ones is called a **language model**.
- A language model captures the **latent space** of language: its statistical structure.
- **Sample** from a language model: feed it an initial string (**conditioning data**) ask it to generate next token.
  - Add the generated output back to the input data, repeat

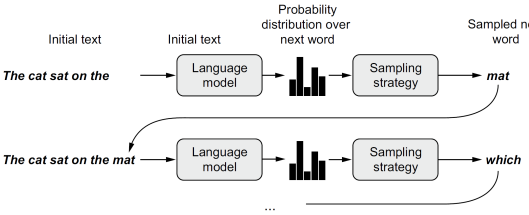

Figure 1: The process of word-by-word text generation using a language model. (Chollet, 2021, pg.367)

The Importance of the Sampling Strategy

- **Greedy sampling** consists of always choosing the most likely next token.
- **Stochastic sampling** sampling from the probability distribution for the next character.
- Useful to **control the amount of randomness** in the sampling process **softmax temperature**

```python
def reweight_distribution(original_distribution, temperature=0.5):
    """Given a softmax probability distribution, reweight the probabilities
    using a temperature variable.  Temperature 0 = greedy sampling, temperature
    1 = uniform like (completely random) distribution

    Parameters
    ----------
    original_distribution : ndarray
        Vector of probability over a vocabulary, probability must sum to 1.0
    temperature : float
        Factor quantifying the entropy of the output distribution

    Returns
    -------
    new_distribution : ndarray
        A reweighted version of the original distribution.
    """
    distribution = np.log(original_distribution) / temperature
    distribution = np.exp(distribution)
    # sum may longer be 1.0, so divide by the sum to obtain new probability
    # distribution that sums to 1.0
    return distribution / np.sum(distribution)
```

EAST TEXAS A&M

Summary

- You can generate discrete sequence data by training a model to predict the next token(s), given previous tokens.
- In the case of text, such a model is called a **language model**. It can be based on either words or characters.
- Sampling the next token requires a balance between adhering to what the model judges likely, and introducing randomness.
- One way to handle this is the notion of `softmax` temperature. Always experiment with different temperatures to find the right one.
- Don't expect to generate meaningful text (even for a chat GPT level of training data), all you are doing is sampling data from a statistical model of which words come after which words. Language models are all form and no substance.

EAST TEXAS A&M

L12.2 DeepDream

CSci 560: Deep Learning w/ Python (Chollet) Ch. 12.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

DeepDream Algorithm

- DeepDream is an artistic image-modification technique that uses the representations learned by convolutional neural networks.
- The DeepDream algorithm is almost identical to the convnet filter-visualization technique introduced previously consisting of running a convnet in reverse:
  - doing gradient ascent on the input to the convnet in order to maximize the activation of a specific filter in an upper layer of the convnet.

The DeepDream algorithm uses this same idea, with a few simple differences:

- With DeepDream, you try to maximize the activation of entire layers rather than that of a specific filter, thus mixing together visualizations of large numbers of features at once.
- You start not from blank, slightly noisy input, but rather from an existing image—thus the resulting effects latch on to preexisting visual patterns, distorting elements of the image in a somewhat artistic fashion.
- The input images are processed at different scales (called octaves), which improves the quality of the visualizations.

EAST TEXAS A&M

Summary

- DeepDream consists of running a convnet in reverse to generate inputs based on the representations learned by the network.
- The results produced are fun and somewhat similar to the visual artifacts induced in humans by the disruption of the visual cortex via psychedelics.
- Note that the process isn't specific to image models or even to convnets. It can be done for speech, music, and more.

EAST TEXAS A&M

L12.3 Neural Style Transfer

CSci 560: Deep Learning w/ Python (Chollet) Ch. 12.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Neural Style Transfer

Figure 2: A style transfer example. (Chollet, 2021, pg.383)

- Another major development in deep-learning image modification **neural style transfer**.
- **Style** essentially means texture, colors and visual patterns.
- The key notion behind implementing style transfer is the same idea that's central to all deep learning algorithms: you define a loss function to specify what you want to achieve, and you minimize this loss.

Neural Style Transfer

We know what we want to achieve: conserving the content of the original image while adopting the style of the reference image.

If we were able to mathematically define **content** and **style** than an appropriate loss function to minimize would be:

```
loss = (distance(style(reference_image) - style(combination_image)) +
        distance(content(original_image) - content(combination_image)))
```

A fundamental observation was that deep convolutional neural networks offer a way to mathematically define the style and content functions

EAST TEXAS A&M

Content Loss and Style Loss

Activations of different layers provide decomposition of contents over different spatial scales.

Content Loss

Expect content of image (global and abstract) to be captured by upper layers.
$\therefore$ A good candidate for content loss is L2 norm between activations of an upper layer in a pretrained convnet, computed over target and generated image.

Style Loss

Content uses an upper layer, but style involves all spatial scales.

$\therefore$ For style, use the inner product of the feature maps of selected layers.

You can use a pretrained convnet to define a loss that will:

- Preserve content by maintaining similar high-level layer activations between the original image and the generated image.
- Preserve style by maintaining similar correlations within activations for both low level layers and high-level layers.

EAST TEXAS A&M

Summary

- Style transfer consists of creating a new image that preserves the contents of a target image while also capturing the style of a reference image.
- Content can be captured by the high-level activations of a convnet.
- Style can be captured by the internal correlations of the activations of different layers of a convnet.
- Hence, deep learning allows style transfer to be formulated as an optimization process using a loss defined with a pretrained convnet.
- Starting from this basic idea, many variants and refinements are possible.

EAST TEXAS A&M

L12.4 Generating Images with Variational Autoencoders

CSci 560: Deep Learning w/ Python (Chollet) Ch. 12.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Sampling from Latent Spaces of Images

- The key idea of image generation is to develop a low-dimensional latent space of representations (a vector space), where any point can be mapped to a "valid" image.
- The module capable of realizing this mapping, is called a **generator** (in context of Generative Adversarial Networks GANs) or a **decoder** (in case of Variable AutoEncoders VAEs).
- GANs and VAEs are two different strategies for learning such latent spaces of image representations, each with its own characteristics.
  - VAEs are great for well structured latent spaces.
  - GANs can generate highly realistic images from latent spaces without as much structure.

EAST TEXAS A&M

Concept Vectors for Image Editing

- **Concept vector** given a latent space of representations, or an embedding space, certain directions in the space may encode interesting axes of variation in the original data.
- For instance, *smile vector* `s` such that if `z` is embedded representation of face
  - `z + s` is the embedded representation of the same face smiling.

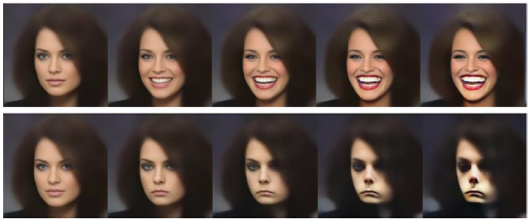

Figure 3: The smile (and frown) vector example. (Chollet, 2021, pg.393)

Variational Autoencoders (VAEs)

- Variational autoencoders are a kind of generative model that's especially appropriate for the task of image editing via concept vectors.
- They're a modern take on autoencoders, a type of network that aims to encode an input to a low-dimensional latent space and then decode it back.
- A classical image autoencoder takes an image, maps it to a latent vector space via an encoder module, and then decodes it back to an output with the same dimensions as the original image, via a decoder module.

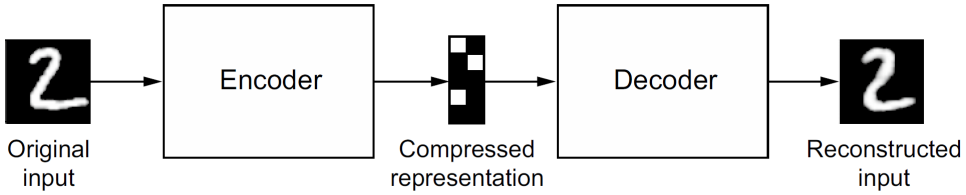

Figure 4: An autoencoder mapping an input x to a compressed representation and then decoding it back as x' (Chollet, 2021, pg.394)

Variational Autoencoder Operation

A VAE, instead of compressing its input image into a fixed code in the latent space, turns the image into the parameters of a statistical distribution: a mean and a variance.

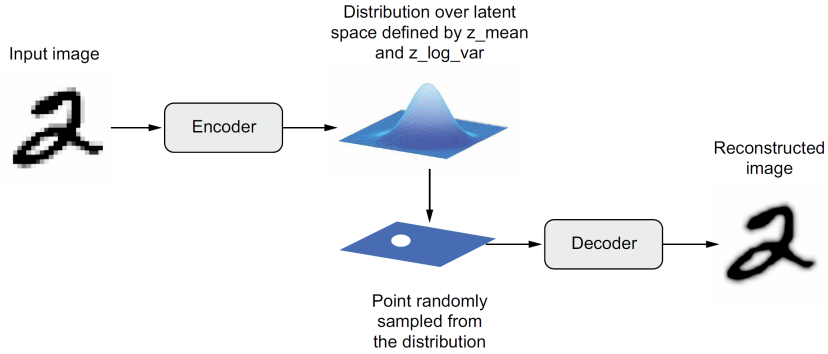

Figure 5: A VAE maps an image to two vectors, z_mean and z_log_sigma, which define a probability distribution over the latent space, used to sample a latent point to decode. (Chollet, 2021, pg.395)

Variational Autoencoder Operation

How a VAE works:

1. An encoder module turns the input sample, `input_img`, into two parameters in a latent space of representations, `z_mean` and `z_log_variance`.
2. You randomly sample a point `z` from the latent normal distribution that's assumed to generate the input image, via `z = z_mean + exp(z_log_variance) * epsilon`, where epsilon is a random tensor of small values.
3. A decoder module maps this point in the latent space back to the original input image.

Continuity, combined with the low dimensionality of the latent space, forces every direction in the latent space to encode a meaningful axis of variation of the data, making the latent space very structured and thus highly suitable to manipulation via concept vectors.

EAST TEXAS A&M

Variational Autoencoder Operation

```
# encodes the input into mean and variance
# parameters
z_mean, z_log_variance = encoder(input_img)

# draws a latent point using a small
#random epsilon
z = z_mean + exp(z_log_variance) * epsilon

# decodes z back to an image
reconstructed_img = decoder(z)

# instantiates the autoencoder model which
# maps an input image to its reconstruction
model = Model(input_img, reconstructed_img)
```

- VAE trained via 2 loss function:
  1. **reconstruction loss** that forces decoded samples to match initial inputs
  2. **regularization loss** that helps learn well-rounded latent distribution.

EAST TEXAS A&M

Summary

- Image generation with deep learning is done by learning latent spaces that capture statistical information about a dataset of images. By sampling and decoding points from the latent space, you can generate never-before-seen images. There are two major tools to do this: VAEs and GANs.
- VAEs result in highly structured, continuous latent representations. For this reason, they work well for doing all sorts of image editing in latent space: face swapping, turning a frowning face into a smiling face, and so on. They also work nicely for doing latent-space-based animations, such as animating a walk along a cross section of the latent space or showing a starting image slowly morphing into different images in a continuous way.
- GANs enable the generation of realistic single-frame images but may not induce latent spaces with solid structure and high continuity.

EAST TEXAS A&M

L12.5 Introduction to Generative Adversarial Networks

CSci 560: Deep Learning w/ Python (Chollet) Ch. 12.5

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Generative Adversarial Networks (GANs)

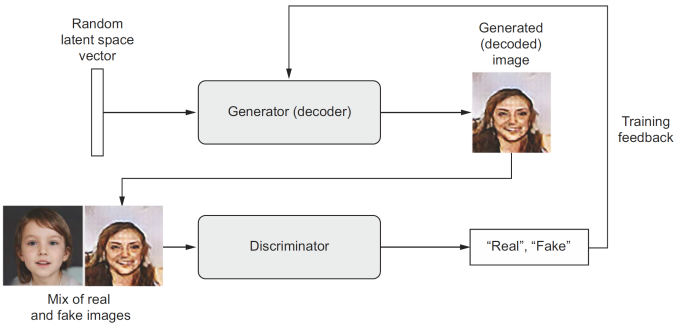

Figure 6: A generator transforms random latent vectors into images, and a discriminator seeks to tell real images from generated ones. The generator is trained to fool the discriminator. (Chollet, 2021, pg.402)

A GAN is: a forger network and an expert network, each being trained to best the other.

As such, a GAN is made of two parts:

- **Generator network** Takes as input a random vector (a random point in the latent space), and decodes it into a synthetic image
- **Discriminator network** (or **adversary**) Takes as input an image (real or synthetic), and predicts whether the image came from the training set or was created by the generator network.

EAST TEXAS A&M

Summary

- A GAN consists of a generator network coupled with a discriminator network. The discriminator is trained to differentiate between the output of the generator and real images from a training dataset, and the generator is trained to fool the discriminator. Remarkably, the generator never sees images from the training set directly; the information it has about the data comes from the discriminator.
- GANs are difficult to train, because training a GAN is a dynamic process rather than a simple gradient descent process with a fixed loss landscape. Getting a GAN to train correctly requires using a number of heuristic tricks, as well as extensive tuning.
- GANs can potentially produce highly realistic images. But unlike VAEs, the latent space they learn doesn't have a neat continuous structure and thus may not be suited for certain practical applications, such as image editing via latentspace concept vectors.

EAST TEXAS A&M

Chapter Summary

- You can use a sequence-to-sequence model to generate sequence data, one step at a time. This is applicable to text generation, but also to note-by-note music generation or any other type of timeseries data.
- DeepDream works by maximizing convnet layer activations through gradient ascent in input space.
- In the style-transfer algorithm, a content image and a style image are combined together via gradient descent to produce an image with the high-level features of the content image and the local characteristics of the style image.
- VAEs and GANs are models that learn a latent space of images and can then dream up entirely new images by sampling from the latent space. Concept vectors in the latent space can even be used for image editing.

EAST TEXAS A&M

Bibliography

Chollet, F. (2021). *Deep learning with python* (second). Manning.

EAST TEXAS A&M